

Tuần 9 — Troubleshooting & Optimization (CloudWatch / X-Ray) + FULL MOCK #1 → HẾT Domain 4

Domain: Domain 4 – Troubleshooting and Optimization (18%) · **Thời lượng:** ~11h (4 buổi) · **Vị trí:** Tuần 1/1 — TRỌN Domain 4; bắt đầu giai đoạn mock

Điều hướng:  Tuần 8 ·  Kế hoạch tổng · Tuần 10 

Mục tiêu tuần này

- **Phân biệt được** CloudWatch (metrics/logs/hiệu năng) vs CloudTrail (audit API call) chỉ trong 5 giây khi đọc keyword đề.
- **Tự tay** bật X-Ray active tracing cho Lambda + API Gateway, thêm annotation và filter trace theo annotation trên service map.
- **Giải thích được** khác biệt ANNOTATIONS (được index → filter được) vs METADATA (không index) trong X-Ray.
- **Tự tay** tạo custom metric (PutMetricData), metric filter từ log, và alarm gắn action SNS.
- **Tạo được** EventBridge rule theo lịch (cron/rate) trigger Lambda.
- **Đọc được** mã lỗi 4xx/5xx và throttling exception, chọn đúng hướng tối ưu (Lambda memory/provisioned concurrency, DynamoDB hot partition/DAX).
- ★ **FULL MOCK #1**: làm 65 câu, canh giờ 130 phút, ghi điểm, review 100% câu sai ngay trong tuần.

Nội dung học chi tiết

Buổi A — Lý thuyết (~3h)

1. CloudWatch Metrics (rất hay hỏi)

- **Namespace + dimension:** metric được gom theo **namespace** (vd **AWS/Lambda**); **dimension** là cặp key-value phân loại metric (vd **FunctionName=abc**). Cùng metric khác dimension = chuỗi số liệu khác nhau.
- **Resolution:**
 - **Standard resolution:** hạt **1 phút** (khi bật **detailed monitoring**) hoặc **5 phút** (mặc định cơ bản, vd EC2).
 - **High-resolution:** tới **1 giây** — dùng khi cần theo dõi cực mịn (custom metric).
- **Custom metric:** đẩy lên bằng API **PutMetricData**. Đây là cách đưa số liệu app (vd số order/phút) vào **CloudWatch**.
- **EMF (Embedded Metric Format):** ghi log có cấu trúc để **CloudWatch** tự trích **metric từ log** — không cần gọi **PutMetricData** riêng, hợp với **Lambda** (log ra là có metric).

2. CloudWatch Logs

- **Log group → log stream:** group gom log theo ứng dụng/nguồn; stream là dòng log của 1 instance/thực thi.
- **Retention:** log **mặc định giữ vô hạn** cho tới khi bạn đặt retention → nhớ set để khỏi tốn tiền.
- **Metric filter:** khớp **pattern** trong log → sinh ra **metric** (vd đếm dòng chứa **ERROR**) → làm nền cho alarm.
- **Subscription filter:** **stream log real-time** tới **Lambda / Kinesis Data Streams / Kinesis Data Firehose** để xử lý/chuyển tiếp.
- **Logs Insights:** ngôn ngữ query để truy vấn/tổng hợp log nhanh (đếm, lọc, thống kê).
- **Athena:** query log/dữ liệu trong **S3** bằng SQL (vd phân tích log **CloudTrail/ALB**).

3. Alarms

- **3 trạng thái:** **OK / ALARM / INSUFFICIENT_DATA** (chưa đủ dữ liệu để đánh giá).
- **Action:** gửi **SNS**, kích **Auto Scaling**, hoặc **EC2 action** (stop/terminate/reboot/recover).
- **Composite alarm:** gộp nhiều alarm bằng logic AND/OR → giảm nhiễu báo động.
- **Metric mặc định của Lambda:** **Invocations, Duration, Errors, Throttles, ConcurrentExecutions**, và **IteratorAge** (với event source dạng **stream** như **Kinesis/DynamoDB Streams** — đo độ trễ xử lý record).

4. CloudWatch vs CloudTrail (phân biệt sống còn)

- **CloudWatch** = metrics + logs + alarms → **giám sát hiệu năng & log**.
- **CloudTrail** = ghi lại **API call**: **AI** gọi API gì, **KHI NÀO**, từ đâu → dùng để **audit / điều tra bảo mật**.
- 🧠 Câu thần chú: "**hiệu năng/log** → **CloudWatch**; **ai làm gì** → **CloudTrail**".

5. EventBridge (CloudWatch Events)

- **Event bus** nhận event; **rule** khớp event theo **event pattern** HOẶC theo **lịch** (schedule **cron/rate**); định tuyến tới **nhiều target** (**Lambda**, **SQS**, **SNS**, **Step Functions**...).
- Kinh điển của đề: "**chạy Lambda định kỳ**" → **EventBridge** rule schedule **cron/rate**.
- **Phân biệt rule (schedule) vs Scheduler**:
 - **EventBridge rule (schedule)**: **cron/rate**, **gắn event bus**.
 - **EventBridge Scheduler** (dịch vụ lịch riêng, 2022): **one-time + recurring**, hỗ trợ **time zone**, **flexible time window**, **270+ service / 6.000+ API** làm target, **không cần event bus**.
- 🧠 Phản xạ: "**lịch chạy có timezone / one-time schedule quy mô lớn**" → **EventBridge Scheduler**.

B Buổi B — Hands-on (~3.5h)

🔪 Lab cầm tay chỉ việc (từng bước + lệnh + code): labs.md.

Lab 1 — Bật **X-Ray** active tracing cho **Lambda** + **API Gateway**, filter theo annotation

1. Trên function **Lambda**: **Configuration** → **Monitoring** → **Active tracing = ON** (bật **active tracing**). Cấp role quyền ghi **X-Ray** (**AWSXRayDaemonWriteAccess**).
2. Trên stage của **API Gateway**: bật **X-Ray Tracing**.
3. Trong code, thêm **annotation** (được index → filter được) và **metadata** (chỉ tham khảo). Ví dụ Node.js:

```
const seg = AWSXRay.getSegment().addNewSubsegment('processOrder');
seg.addAnnotation('orderType', 'PREMIUM'); // INDEXED → filter được
seg.addMetadata('rawPayload', payload);    // KHÔNG index
seg.close();
```

4. Gọi API vài lần với các **orderType** khác nhau.

5. Mở **Service Map** → xem sơ đồ **API GW** → **Lambda** (→ downstream). Vào **Traces**, dùng filter expression **annotation.orderType = "PREMIUM"** → chỉ ra đúng các trace khớp. **Thử filter theo metadata** → **KHÔNG được** (đây là câu hỏi công).

 **Ghi chú (X-Ray hợp nhất dưới CloudWatch):** từ 11/2024 trải nghiệm **X-Ray** được đưa vào **CloudWatch** console (xem trace trong CloudWatch), kèm **CloudWatch Application Signals** (APM) và **Transaction Search**. Khái niệm segment/subsegment/annotation/sampling vẫn nguyên.

Lab 2 — Custom metric + metric filter từ log + alarm

1. Đẩy 1 **custom metric** bằng **PutMetricData**:

```
aws cloudwatch put-metric-data --namespace "MyApp" \
  --metric-name OrdersProcessed --value 1 --unit Count
```

2. Tạo **metric filter** trên log group của app: pattern **ERROR** → metric **AppErrorCount** (namespace **MyApp**).

```
aws logs put-metric-filter --log-group-name /aws/lambda/my-fn \
  --filter-name errors --filter-pattern "ERROR" \
  --metric-transformations
metricName=AppErrorCount,metricNamespace=MyApp,metricValue=1
```

3. Tạo **alarm** trên **AppErrorCount**: ví dụ **>= 5** trong 1 chu kỳ → **action** publish tới topic **SNS**.

```
aws cloudwatch put-metric-alarm --alarm-name too-many-errors \
  --namespace MyApp --metric-name AppErrorCount \
  --statistic Sum --period 300 --threshold 5 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --evaluation-periods 1 --alarm-actions <sns-topic-arn>
```

4. Ghi log chứa **ERROR** đủ số lần → alarm chuyển **OK** → **ALARM** → nhận mail **SNS**.

Lab 3 — EventBridge rule cron → Lambda

1. Tạo rule theo lịch (chạy mỗi 5 phút):

```
aws events put-rule --name every-5-min --schedule-expression "rate(5 minutes)"
```

2. Gắn target là **Lambda**:

```
aws events put-targets --rule every-5-min \
  --targets "Id"="1", "Arn"="<lambda-arn>"
```

3. Thêm permission cho **EventBridge** gọi **Lambda** (`aws lambda add-permission` với `--principal events.amazonaws.com`). Quan sát log để thấy function chạy đều đặn.

C Buổi C — Bổ sung (~2.5h)

Đọc & xử lý mã lỗi

Mã	Loại	Ý nghĩa / xử lý
4xx	Lỗi client	Sai request. 400 bad request; 403 không có quyền; 429 = throttling / too many requests
5xx	Lỗi server	500 internal; 502 bad gateway; 503 service unavailable
Throttle DynamoDB	client	ProvisionedThroughputExceededException
Throttle chung	—	Retry với exponential backoff + jitter (SDK thường tự làm)

Optimization — Lambda

- **Memory → CPU**: tăng memory thì **CPU tăng theo tỉ lệ** → hàm chạy nhanh hơn; đôi khi **rẻ hơn** vì thời gian chạy giảm bù cho giá memory cao hơn.
- **Tìm điểm tối ưu**: dùng **AWS Lambda Power Tuning** để chọn cấu hình memory cho ra chi phí/tốc độ tốt nhất.
- **Cold start nhiều**: dùng **Provisioned Concurrency** để giữ sẵn môi trường → loại cold start. Ngoài ra: **connection reuse** (khởi tạo SDK/DB client ngoài handler để tái dùng qua các lần gọi).

Optimization — DynamoDB & khác

- **Hot partition:** phân tán **partition key** cho đều → tránh dồn traffic vào 1 partition (nguyên nhân chính gây throttle).
- Cân nhắc **on-demand** khi traffic khó đoán; dùng **DAX** cho **read nặng** (cache in-memory, giảm độ trễ về micro giây).
- **SQS batching** để giảm số request; **caching layer** (**ElastiCache**/DAX/**API Gateway** cache) giảm tải downstream.

Đọc thêm: **X-Ray** concepts (segment/subsegment, sampling), **CloudWatch** Logs Insights query syntax, **EventBridge** schedule expression.

D Buổi D — Practice + Review (~2h)

 **Bộ câu hỏi luyện tập của tuần:** [questions.md](#) — đáp án & giải thích: [answers.md](#). (bằng tiếng Anh — văn phong đề thật để làm quen đề.)

- ★★ **FULL MOCK #1** — 65 câu, canh giờ **CHÍNH XÁC 130 phút**. Làm như thi thật: không tra cứu, không dừng giữa chừng.
- **Ghi điểm** ngay, tính % theo từng Domain.
- **Review 100% câu sai NGAY trong tuần** (không để dồn): với mỗi câu ghi lý do sai (không biết / đọc nhầm / bẫy) vào **sổ câu sai**.
- **Spaced repetition:** ôn flashcard số liệu Domain 4 theo mốc **1 / 3 / 7 ngày** (**X-Ray** port, resolution metric, Lambda metrics).

PHẢI NHỚ tuần này

Fact	Con số / Ghi nhớ
X-Ray segment vs subsegment	segment = dữ liệu 1 service; subsegment = chi tiết nhỏ hơn (vd 1 lời gọi downstream)
ANNOTATIONS	cặp key-value ĐƯỢC INDEX → filter/query được trace
METADATA	KHÔNG index → chỉ tham khảo, không filter được
X-Ray sampling	sampling rules giảm số trace → tiết kiệm chi phí
X-Ray daemon	nghe UDP cổng 2000 , gom & gửi trace lên X-Ray
Bật X-Ray cho Lambda	bật active tracing

Fact	Con số / Ghi nhớ
CloudWatch vs CloudTrail	hiệu năng/log → CloudWatch; ai gọi API gì/khi nào → CloudTrail
Metric resolution	standard 1 phút (detailed) / 5 phút; high-resolution tới 1 giây
Custom metric	đẩy bằng PutMetricData ; hoặc dùng EMF để trích từ log
Metric filter vs subscription filter	metric filter = tạo metric từ pattern log; subscription filter = stream log tới Lambda/Kinesis/Firehose
Alarm states	OK / ALARM / INSUFFICIENT_DATA ; action: SNS / Auto Scaling / EC2
Metric mặc định Lambda	Invocations, Duration, Errors, Throttles, ConcurrentExecutions, IteratorAge (stream)
EventBridge rule	event pattern HOẶC lịch cron/rate ; nhiều target
Mã lỗi	4xx = client (400/403/429 throttling), 5xx = server (500/502/503)
DynamoDB throttle	ProvisionedThroughputExceededException → retry exponential backoff + jitter
Lambda memory	tăng memory → CPU tăng tỉ lệ ; provisioned concurrency loại cold start

⚠ Bẫy đề hay gặp

- Thấy "muốn **FILTER/tìm trace** theo giá trị (vd userId)" → chọn nhầm **metadata** → đúng là **annotation** (chỉ annotation được index).
- Thấy "ai đã gọi API xóa bucket / thay đổi cấu hình" → tưởng CloudWatch → đúng là CloudTrail (audit API).
- Thấy "giám sát CPU/độ trễ/đếm lỗi" → tưởng CloudTrail → đúng là CloudWatch metrics/logs.
- Thấy "Lambda cold start nhiều, cần độ trễ ổn định" → tưởng tăng memory là đủ → đúng là **Provisioned Concurrency**.
- Thấy "tăng memory Lambda" mà nghĩ chỉ tốn thêm tiền → thực ra **CPU tăng theo tỉ lệ**, có thể **rẻ hơn** vì chạy nhanh hơn.

- Thấy "**DynamoDB** bị throttle dù còn capacity" → tưởng thiếu RCU/WCU → thường là **hot partition** (partition key phân tán kém).
- Thấy "chạy job theo lịch" → chọn nhầm **Lambda** cron nội bộ → đúng là **EventBridge schedule (cron/rate)**.
- Thấy "stream log tới hệ thống khác xử lý real-time" → chọn nhầm metric filter → đúng là **subscription filter**.
- Thấy lỗi **429** → nhầm là lỗi server → thực ra là **client throttling** → **retry backoff + jitter**.



Phản xạ nhanh (keyword → đáp án)

Thấy từ khoá	Bật ngay
filter/tìm trace theo giá trị	annotation (indexed)
dữ liệu kèm theo chỉ để xem, không filter	metadata
ai gọi API gì / khi nào / audit	CloudTrail
CPU/độ trễ/đếm lỗi/log ứng dụng	CloudWatch
đẩy số liệu app tùy chỉnh lên	PutMetricData (hoặc EMF)
tạo metric từ pattern trong log	metric filter
đẩy log real-time sang Lambda/Kinesis/Firehose	subscription filter
query/tổng hợp log nhanh	Logs Insights
báo động chưa đủ dữ liệu	INSUFFICIENT_DATA
chạy Lambda /task theo lịch định kỳ	EventBridge schedule (cron/rate)
gom & gửi trace, UDP 2000	X-Ray daemon
bật X-Ray cho Lambda	active tracing
cold start nhiều	Provisioned Concurrency
tìm cấu hình memory tối ưu	AWS Lambda Power Tuning
DynamoDB read nặng cần micro giây	DAX

DynamoDB throttle dù còn capacity

hot partition (phân tán partition key)

lỗi 429 /

ProvisionedThroughputExceededException

retry exponential backoff + jitter



Lab checklist

- ☐ Bật **X-Ray** active tracing cho **Lambda** + **API Gateway**, xem service map.
- ☐ Thêm annotation + metadata; filter trace theo annotation (và xác nhận metadata KHÔNG filter được).
- ☐ Đẩy custom metric bằng **PutMetricData**.
- ☐ Tạo metric filter từ log + alarm gắn action **SNS**, kích được trạng thái **ALARM**.
- ☐ Tạo **EventBridge** rule **cron/rate** trigger **Lambda**, xác nhận chạy định kỳ.
- ☐ Làm **FULL MOCK #1** (65 câu / 130 phút), ghi điểm, review 100% câu sai.



Cổng tự kiểm tra (phải trả lời trôi chảy mới sang tuần sau)

- Muốn **FILTER** trace theo 1 giá trị → dùng annotation hay metadata? Đáp án gọn: **annotation** (được index nên query/filter được); metadata không index, chỉ để tham khảo.
- CloudWatch** khác **CloudTrail** chỗ nào? Đáp án gọn: **CloudWatch** = metrics/logs/alarms (hiệu năng & log); **CloudTrail** = audit **ai gọi API gì, khi nào**.
- Lambda** cold start nhiều → tối ưu bằng gì? Đáp án gọn: **Provisioned Concurrency** (giữ sẵn môi trường); kèm connection reuse ngoài handler.
- Tăng memory **Lambda** ảnh hưởng gì tới CPU/chi phí? Đáp án gọn: CPU tăng theo tỉ lệ → chạy nhanh hơn; có thể **rẻ hơn** vì thời gian giảm; dùng Power Tuning tìm điểm tối ưu.
- X-Ray** daemon nghe cổng nào, làm gì? Đáp án gọn: **UDP cổng 2000**, gom trace từ SDK và gửi lên **X-Ray**.
- DynamoDB** throttle dù còn capacity → nguyên nhân & xử lý? Đáp án gọn: **hot partition** → phân tán partition key; lỗi

`ProvisionedThroughputExceededException` → retry backoff + jitter; cân nhắc on-demand / DAX.

- ★ **FULL MOCK #1**: đã làm **65 câu / 130 phút** canh giờ, ghi điểm và **review 100% câu sai** chưa? Nếu chưa → chưa được coi là xong Domain 4.

Tài nguyên tuần này

 **Đã crawl sẵn tài liệu AWS vào `resources/`** — đọc offline được.

- AWS Docs: **Amazon CloudWatch** User Guide — metrics (namespace/dimension/resolution), custom metrics (**PutMetricData**), EMF.
- AWS Docs: **Amazon CloudWatch Logs** — log group/stream, retention, metric filter, subscription filter, Logs Insights.
- AWS Docs: **Amazon CloudWatch** Alarms — states, actions, composite alarms; **AWS Lambda** metrics reference.
- AWS Docs: **AWS CloudTrail** User Guide — API activity logging (so sánh với **CloudWatch**).
- AWS Docs: **Amazon EventBridge** User Guide — rules (event pattern / schedule **cron/rate**), targets.
- AWS Docs: **AWS X-Ray** Developer Guide — segments/subsegments, annotations vs metadata, sampling, service map, daemon, active tracing.
- AWS Docs: **AWS Lambda** — performance optimization; **AWS Lambda Power Tuning** (GitHub); **Amazon DynamoDB** best practices (partition key design, DAX).
- Khoá học: Stephane Maarek — mục Monitoring (**CloudWatch/X-Ray/CloudTrail/EventBridge**) + Optimization; Adrian Cantrill — Observability & performance.

Checklist hoàn thành Tuần 9

- ☐ Hoàn thành 4 buổi A/B/C/D
- ☐ Thuộc lòng bảng "PHẢI NHỚ" (annotation vs metadata, resolution, Lambda metrics, mã lỗi)
- ☐ Phân biệt được **CloudWatch** vs **CloudTrail** qua keyword
- ☐ Hoàn thành 5 lab (X-Ray tracing, custom metric, metric filter + alarm, EventBridge cron)
- ☐ **Làm FULL MOCK #1** (65 câu / 130 phút), ghi điểm, review 100% câu sai — KẾT THÚC Domain 4

- ☐ Vượt Cổng tự kiểm tra